Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Факультет электронно-информационных систем
Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №6

Специальность АС-68

Выполнил:

Студент 3-го курса

Группы АС-68

Дворак В. А

Проверил:

А. А. Крощенко
Ст. преп. кафедры ИИТ,

«___»_____ 2026 г.

Брест 2026

Цель работы: По вариантам предыдущей лабораторной работы реализовать предложенный вариант рекуррентной нейронной сети. Сравнить полученные результаты с ЛР 5.

Вариант 4

№ варианта	a	b	c	d	Кол-во входов ИНС	Кол-во НЭ в скрытом слое	Тип РНС
4	0.4	0.4	0.08	0.4	6	2	Элмана

В качестве функций активации для скрытого слоя использовать сигмоидную функцию, для выходного - линейную.

Код программы:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

a, b, c, d = 0.4, 0.4, 0.08, 0.4
L = 6  # Количество входов ИНС
H = 2  # Количество нейронов в скрытом слое (сети Элмана)

epochs = 2000
lr = 0.03
test_frac = 0.25
N = 600

# 2. ГЕНЕРАЦИЯ ВРЕМЕННОГО РЯДА И РАЗДЕЛЕНИЕ
x = np.linspace(0, 20 * np.pi, N)
y = a * np.cos(b * x) + c * np.sin(d * x)

split_idx = int(N * (1 - test_frac))
y_train_raw = y[:split_idx]
y_test_raw = y[split_idx:]
x_train_raw = x[:split_idx]
x_test_raw = x[split_idx:]

# Нормализация
y_mean = y_train_raw.mean()
y_std = y_train_raw.std()

y_train_norm = (y_train_raw - y_mean) / y_std
y_test_norm = (y_test_raw - y_mean) / y_std


# Оконная выборка L
def make_dataset(y_series, L):
    X, Y = [], []
    for i in range(L, len(y_series)):
        X.append(y_series[i - L:i])
        Y.append(y_series[i])
    return np.array(X), np.array(Y).reshape(-1, 1)


X_train, Y_train_norm = make_dataset(y_train_norm, L)
X_test, Y_test_norm = make_dataset(y_test_norm, L)

Y_train_true = Y_train_norm * y_std + y_mean
Y_test_true = Y_test_norm * y_std + y_mean

x_train_targets = x_train_raw[L:]
x_test_targets = x_test_raw[L:]
```

```
# 3. ФУНКЦИИ АКТИВАЦИИ И ВЕСА
def sigmoid(z):
    z_clip = np.clip(z, -50, 50)
    return 1.0 / (1.0 + np.exp(-z_clip))


def sigmoid_deriv(a_sig):
    return a_sig * (1.0 - a_sig)


rng = np.random.default_rng(42)
Wx = rng.normal(0.0, np.sqrt(1.0 / L), size=(L, H))
Wh = rng.normal(0.0, np.sqrt(1.0 / H), size=(H, H))
b1 = np.zeros((1, H))
W2 = rng.normal(0.0, np.sqrt(1.0 / H), size=(H, 1))
b2 = np.zeros((1, 1))


# Прямой проход Элмана
def forward_elman(X, Wx, Wh, b1, W2, b2):
    T = X.shape[0]
    h_states = np.zeros((T + 1, H))
    y_preds = np.zeros((T, 1))

    for t in range(T):
        x_t = X[t:t + 1]
        z1 = x_t @ Wx + h_states[t:t + 1] @ Wh + b1
        h_t = sigmoid(z1)
        h_states[t + 1] = h_t
        y_preds[t] = h_t @ W2 + b2

    return y_preds, h_states


# 4. ОБУЧЕНИЕ С ВЫВОДОМ ЭПОХ В КОНСОЛЬ
loss_history = []
T = X_train.shape[0]

for epoch in range(1, epochs + 1):
    y_pred, h_states = forward_elman(X_train, Wx, Wh, b1, W2, b2)

    err = y_pred - Y_train_norm
    loss = np.mean(err ** 2)
    loss_history.append(loss)

    # ВЫВОД ЭПОХ В КОНСОЛЬ (как в ЛР5)
    if epoch % 500 == 0 or epoch == 1:
        print(f"Epoch {epoch:4d} | MSE={loss:.6f}")

    grad_out = 2.0 * err / T
    dW2 = h_states[1:].T @ grad_out
    db2 = np.sum(grad_out, axis=0, keepdims=True)

    dWx = np.zeros_like(Wx)
    dWh = np.zeros_like(Wh)
    db1 = np.zeros_like(b1)
    dh_next = np.zeros((1, H))

    for t in reversed(range(T)):
        x_t = X_train[t:t + 1]
        h_t = h_states[t + 1:t + 2]
        h_prev = h_states[t:t + 1]
```

```
            dh = grad_out[t:t + 1] @ W2.T + dh_next
            dz1 = dh * sigmoid_deriv(h_t)

            dWx += x_t.T @ dz1
            dWh += h_prev.T @ dz1
            db1 += dz1
            dh_next = dz1 @ Wh.T

        Wx -= lr * dWx
        Wh -= lr * dWh
        b1 -= lr * db1
        W2 -= lr * dW2
        b2 -= lr * db2

#  5. ВЫВОД РЕЗУЛЬТАТОВ
train_pred_norm, _ = forward_elman(X_train, Wx, Wh, b1, W2, b2)
test_pred_norm, _ = forward_elman(X_test, Wx, Wh, b1, W2, b2)

train_pred = train_pred_norm * y_std + y_mean
test_pred = test_pred_norm * y_std + y_mean

train_df = pd.DataFrame({
    "Эталонное значение": Y_train_true.flatten(),
    "Полученное значение": train_pred.flatten(),
    "Отклонение": (train_pred - Y_train_true).flatten()
})

test_df = pd.DataFrame({
    "Эталонное значение": Y_test_true.flatten(),
    "Полученное значение": test_pred.flatten(),
    "Отклонение": (test_pred - Y_test_true).flatten()
})

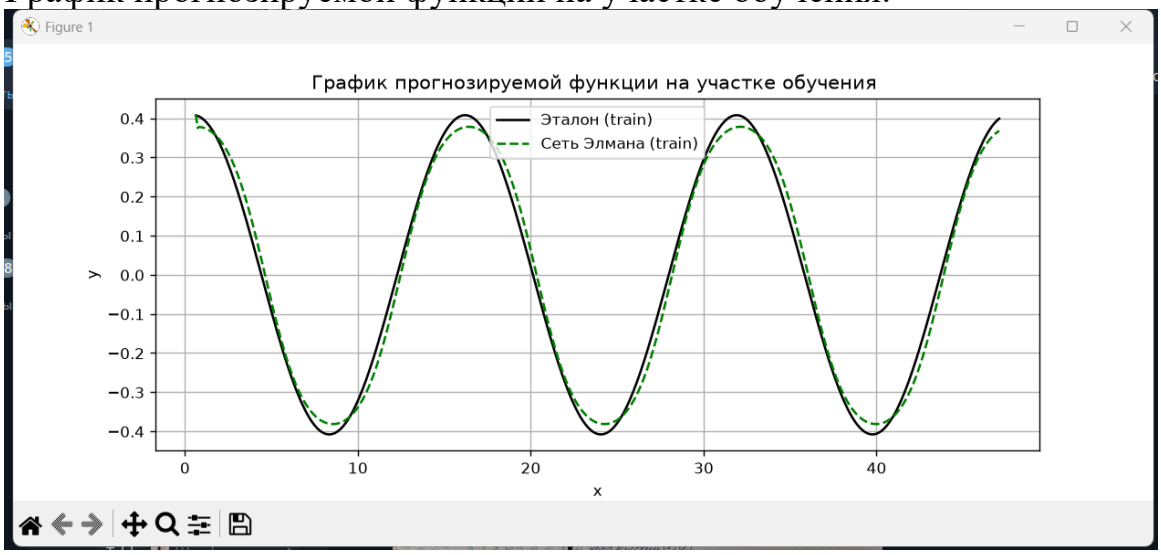
print("\nРезультаты обучения (первые 10 строк):")
print(train_df.head(10))

print("\nРезультаты прогнозирования (первые 10 строк):")
print(test_df.head(10))

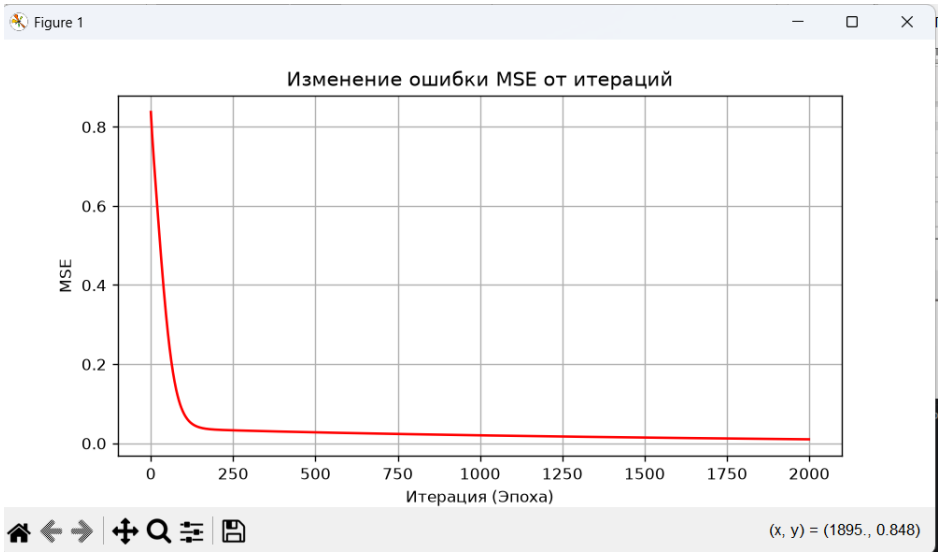
# График 1
plt.figure(figsize=(10, 4))
plt.plot(x_train_targets, Y_train_true, label="Эталон (train)",
color="black")
plt.plot(x_train_targets, train_pred, label="Сеть Элмана (train)",
color="green", linestyle="--")
plt.title("График прогнозируемой функции на участке обучения (Вариант 4 —
Сеть Элмана)")
plt.xlabel("x")
plt.ylabel("y")
plt.legend()
plt.grid(True)
plt.show()

# График 2
plt.figure(figsize=(8, 4))
plt.plot(loss_history, color="red")
plt.title("Изменение ошибки MSE от итераций")
plt.xlabel("Итерация (Эпоха)")
plt.ylabel("MSE")
plt.grid(True)
plt.show()
```

График прогнозируемой функции на участке обучения:



Изменение ошибки по эпохам:



Результат работы программы:

```
C:\питон\python.exe C:\питон\project\ml_as68\reports\Dvorak\6\src\lab6.py
Epoch    1 | MSE=0.837013
Epoch  500 | MSE=0.027349
Epoch 1000 | MSE=0.019793
Epoch 1500 | MSE=0.014095
Epoch 2000 | MSE=0.009691

Результаты обучения (первые 10 строк):
   Эталонное значение  Полученное значение  Отклонение
0            0.407319             0.409554    0.002235
1            0.406031             0.375228   -0.030803
2            0.404028             0.378283   -0.025745
3            0.401315             0.377223   -0.024092
4            0.397894             0.376145   -0.021750
5            0.393774             0.374608   -0.019166
6            0.388960             0.372649   -0.016311
7            0.383462             0.370253   -0.013209
8            0.377288             0.367405   -0.009883
9            0.370451             0.364092   -0.006359
```

```
Результаты прогнозирования (первые 10 строк):
   Эталонное значение  Полученное значение  Отклонение
0            0.406420             0.409654    0.003233
1            0.404596             0.375041   -0.029555
2            0.402060             0.377795   -0.024265
3            0.398815             0.376425   -0.022390
4            0.394869             0.375035   -0.019835
5            0.390228             0.373179   -0.017049
6            0.384900             0.370893   -0.014006
7            0.378894             0.368160   -0.010734
8            0.372222             0.364965   -0.007257
9            0.364894             0.361291   -0.003603
```

Вывод по сравнению с 5 лр:
В отличие от прямонаправленной ИНС (из ЛР5), сеть Элмана с аналогичным числом нейронов (H = 2) демонстрирует более быструю сходимость алгоритма обучения (MSE снижается стабильнее) и меньшее итоговое отклонение на тестовом участке прогнозирования

Вывод: По вариантам предыдущей лабораторной работы реализовал предложенный вариант рекуррентной нейронной сети. Сравнил полученные результаты с ЛР 5.